

Uppsala University
Department of Information Technology

Literature Seminar

Written Preparation Report

Course: Data Engineering II

Arnab Kumar Ghosh
Eric Buxton
Prodip Kumar Das
Rahul Bhat

May 13, 2026

Contents

1	Contextualization and Orchestration: Meaning, Importance, and Available Tools	2
2	Serverless Architecture, Digitalization of Society, and Big Data	2
3	Model Serving and CI/CD in Data Engineering	3
4	Distributed vs. Federated Machine Learning	3

1 Contextualization and Orchestration: Meaning, Importance, and Available Tools

Role and Importance

Contextualization is the automated process of setting up a new virtual machine or container so it can start working correctly [1]. When cloud systems scale up, they create new copies from a standard template. Contextualization injects specific details like IP addresses and security keys into these copies so they have their own identity [1].

Orchestration is the broader management of these parts. It coordinates how different services work together across many computers and locations [6]. It is important because it automates complex tasks like deploying software and managing resources, which reduces human error and keeps the system running smoothly [1, 6].

Four Frameworks and Tools

1. **Kubernetes (Orchestration):** This tool manages containerized applications. It automatically handles scaling and ensures that if one part fails, another starts immediately to keep the service online [4].
2. **Puppet and Chef (Configuration Management):** These tools automate the setup and maintenance of software on many virtual machines at once. They make sure every machine follows the same rules and stays updated [1].
3. **VM Contextualizer (Contextualization):** This is a specific tool that prepares virtual machine images by injecting custom data during boot time. It helps keep the main image simple while allowing each copy to be unique [1].
4. **TOSCA (Orchestration):** This is a standard way to describe how a cloud application should be built. It tells orchestration tools exactly what hardware and connections are needed to run a complex application [6].

2 Serverless Architecture, Digitalization of Society, and Big Data

Contemporary serverless architecture has become an important part of the digitalization of society because it allows organizations to create and run scalable digital services without having to manage physical servers or complicated infrastructure themselves [2]. In serverless computing, cloud providers automatically handle resources and run functions only when they are needed. This approach supports modern services such as online banking, e-commerce, IoT systems, streaming platforms, and AI applications, where user activity can change quickly. Since companies only pay for the resources they actually use, serverless computing can reduce costs and make innovation easier and more accessible [2, 5].

Serverless architecture is also strongly connected to Big Data applications. Today's Big Data systems generate huge amounts of real-time information from mobile devices, sensors, social media, and online transactions [3]. Serverless platforms can automatically scale up or down to process this data efficiently. Technologies such as AWS Lambda, Google Cloud Functions, and Azure Functions are often combined with Big Data tools like Apache Kafka, Spark, and cloud storage systems to support real-time analytics and event-driven data processing [2, 5]. Because of this, serverless solutions are especially useful for recommendation systems, fraud detection, and smart city technologies.

Another major benefit is faster software development and deployment. Developers can spend more time focusing on the application itself instead of maintaining servers, which improves

flexibility and supports continuous updates [5]. At the same time, serverless systems also have some disadvantages, such as startup delays, dependence on specific cloud providers, and difficulties when debugging distributed systems [2]. Even with these challenges, serverless computing is increasingly seen as an important technology for supporting both the digital transformation of society and the growing need for scalable Big Data processing.

3 Model Serving and CI/CD in Data Engineering

Role and Importance

Training a model is only the first step. To be useful, the model must run as a live service so other applications can send data and receive predictions — this is model serving. It manages model versions, routes requests, and scales with demand. CI/CD (Continuous Integration and Continuous Deployment) automates the steps between updating a model and deploying it to production. Because data changes over time, models need regular retraining and redeployment. Without CI/CD, this is slow and error-prone. Together, model serving and CI/CD let teams update models quickly, catch errors through automated testing, and keep predictions stable in production [7, 8].

Four Frameworks

1. **TensorFlow Serving (Model Serving):** An open-source tool by Google for deploying TensorFlow models. It serves predictions via REST or gRPC APIs and supports multiple model versions at once, which is useful for A/B testing. It is built for high-speed, high-load environments.
2. **MLflow Model Serving (Model Serving):** An open-source platform that works with any ML framework. A single command deploys a trained model as a REST API. It also provides a model registry to track versions and safely move models from testing to production.
3. **Jenkins (CI/CD):** One of the most widely used CI/CD tools. It can automatically retrain a model when new data arrives, run validation tests, and deploy passing models. Its many plugins connect it to cloud platforms, Docker, and ML frameworks.
4. **PerfCI (CI/CD):** A CI/CD framework designed specifically for ML pipelines. It automatically benchmarks model performance such as speed and accuracy. Each time a change is made, it flags regressions before the model reaches production. This makes it especially useful for teams that need to ensure model quality does not drop between updates.

4 Distributed vs. Federated Machine Learning

References

- [1] D. Armstrong et al. "Contextualization: dynamic configuration of virtual machines". In: *Journal of Cloud Computing: Advances, Systems and Applications* 4.17 (2015).
- [2] Ioana Baldini et al. "Serverless Computing: Current Trends and Open Problems". In: *Research Advances in Cloud Computing* (2017), pp. 1–20.
- [3] Min Chen, Shiwen Mao, and Yunhao Liu. "Big Data: A Survey". In: *Mobile Networks and Applications* 19.2 (2014), pp. 171–209.
- [4] S. Hariri and M. Carrasco Kind. "Batch and online anomaly detection for scientific applications in a Kubernetes environment". In: *Proceedings of the 8th International Workshop on Cloud Data and Platforms (ScienceCloud '18)*. 2018.

- [5] Eric Jonas et al. “Cloud Programming Simplified: A Berkeley View on Serverless Computing”. In: *arXiv preprint arXiv:1902.03383* (2019).
- [6] B. Qian et al. “Orchestrating the Development Lifecycle of Machine Learning-based IoT Applications: A Taxonomy and Survey”. In: *ACM Computing Surveys* 53.4 (2020).
- [7] Bin Qian et al. “Orchestrating the Development Lifecycle of Machine Learning-based IoT Applications: A Taxonomy and Survey”. In: *ACM Computing Surveys* 54.1 (2020), pp. 1–48.
- [8] Joost Verbraeken et al. “A Survey on Distributed Machine Learning”. In: *ACM Computing Surveys* 53.2 (2020), pp. 1–33.